Category A Practice Problems

PROBLEM 1 • Basic

Checkout Payment Handler

Problem Statement

An online checkout system accepts multiple ways to pay. Every payment needs a way to process itself and a permanent transaction ID, but how a payment is actually processed is genuinely different for a credit card versus cash — there's no such thing as a generic, undefined 'payment' with no specific way of paying.

Requirements

- PaymentMethod must be an abstract class with an abstract method processPayment(double amount) that returns a confirmation message. It must also provide real, shared behavior: a final transactionId field assigned via a shared static counter inside its own constructor, common to every subclass.
- PaymentMethod must also declare an overloaded processPayment(double amount, String note) — true compile-time polymorphism, sharing the same name but a different parameter list, where the note version appends the note to whatever the single-argument version already produces.
- CreditCardPayment and CashPayment must both extend PaymentMethod directly and implement processPayment(double amount) with genuinely different logic.
- Confirm, through your design, that new PaymentMethod() cannot compile — only a specific way of paying can ever be constructed.
- Write a static method that accepts a PaymentMethod reference and prints its confirmation message, without checking which subclass it actually is. Inside a small test method, store a CreditCardPayment in a PaymentMethod-typed variable and note in a comment that this assignment is upcasting.

Function Signature(s)

```
public abstract class PaymentMethod  // assigns a final transactionId internally
public abstract String processPayment(double amount);
String processPayment(double amount, String note)  // overloaded, defined once on PaymentMethod
public String getTransactionId()
public CreditCardPayment(String cardNumberLastFour)
public CashPayment()
static void printConfirmation(PaymentMethod payment, double amount)
```

Examples

Input	Output
`CreditCardPayment cc = new CreditCardPayment("4471"); cc.processPayment(250.0)`	`"Charged $250.0 to card ending 4471 - Txn TXN-1001"`

Input	Output
`CashPayment cash = new CashPayment(); cash.processPayment(40.0)`	`"Received $40.0 in cash - Txn TXN-1002"`

Explanation: *The static counter advances once per PaymentMethod object regardless of which subclass actually constructed it.*

Input	Output
`cc.processPayment(250.0, "Birthday gift")`	`"Charged $250.0 to card ending 4471 - Txn TXN-1001 (Birthday gift)"`

Explanation: *This is the overloaded version — same method name, one extra parameter, resolved at compile time based on how many arguments were passed.*

Input	Output
`PaymentMethod ref = cc; // upcasting: a CreditCardPayment reference stored as its parent type printConfirmation(ref, 250.0)`	`"Charged $250.0 to card ending 4471 - Txn TXN-1001"`

HINT Put the static counter increment and the transactionId assignment inside PaymentMethod's own constructor — every subclass reaches it via super(), so it only has to exist in one place. For the overload, let the two-argument version simply call the one-argument version internally and glue the note onto the end.

CONSTRAINTS: amount is always positive.

Key Concepts Tested: Abstract classes and abstract methods, single inheritance, final fields via a static counter, compile-time polymorphism via method overloading, upcasting, runtime polymorphic dispatch, why an abstract class can never be instantiated directly.

PROBLEM 2 • Basic → Intermediate

Home Safety Alert Network

Problem Statement

A home safety app wants any device capable of raising an alert to plug into one shared alert-broadcasting system — regardless of whether that device is a full security sensor, or something else entirely with no relation to sensors at all.

Requirements

- Define an interface Alertable with a single method sendAlert(String message) returning a String confirmation.
- SecuritySensor must be a concrete base class with an instance field zoneName and a method getZoneName(); MotionSensor must extend SecuritySensor AND implement Alertable, overriding sendAlert() with @Override.
- DualZoneMotionSensor must extend MotionSensor — a genuine multilevel hierarchy three classes deep — adding a second zone name and overriding sendAlert() again; its override must call super.sendAlert() and append the second zone, rather than rebuilding the message from scratch.
- SmokeDetector must implement Alertable directly, with no relationship whatsoever to SecuritySensor — proving Alertable works for devices that share no ancestry at all.
- Write a static method broadcastAll(Alertable[] devices, String message) that loops through a mixed array and calls sendAlert() on each, regardless of what kind of object it actually is.
- Separately, write a method that accepts an Alertable reference, checks with instanceof whether it's actually a MotionSensor, and if so, safely downcasts it to read its zoneName.

Function Signature(s)

```
public interface Alertable { String sendAlert(String message); }
public SecuritySensor(String zoneName)
public MotionSensor(String zoneName)
public DualZoneMotionSensor(String zoneName, String secondZoneName)
public SmokeDetector(String deviceId)
static void broadcastAll(Alertable[] devices, String message)
static String getZoneIfMotionSensor(Alertable a)
```

Examples

Input	Output
`MotionSensor m = new MotionSensor("Living Room");` `m.sendAlert("Motion detected")`	`"[Living Room] Motion detected"`

Input	Output

`DualZoneMotionSensor d = new`
`DualZoneMotionSensor("Hallway", "Stairwell");`
`d.sendAlert("Motion detected")`
	`"[Hallway] Motion detected [also covering Stairwell]"`

Explanation: *DualZoneMotionSensor reuses MotionSensor's own message via super.sendAlert(message) and appends its second zone, rather than rebuilding the whole message itself — the same super-reuse pattern works no matter how many levels deep the override sits.*

Input	Output
`SmokeDetector s = new SmokeDetector("SD-01");` `s.sendAlert("Smoke detected")`	`"[SD-01] Smoke detected"`

Input	Output
`getZoneIfMotionSensor(m)` `getZoneIfMotionSensor(s)`	`"Living Room"` `"Not a motion sensor"`

HINT SmokeDetector implementing Alertable with zero connection to SecuritySensor is exactly the point — a capability interface never requires shared ancestry, only a shared promise. For DualZoneMotionSensor, call super.sendAlert(message) first and only add the new detail on top, the same way you'd reuse a parent's work at any other level.

CONSTRAINTS: message is never blank.

Key Concepts Tested: Interfaces as pure capability contracts, hierarchical and multilevel inheritance combined, method overriding with super reuse across multiple levels, a class implementing an interface without extending anything related, polymorphic dispatch over an interface-typed array, safe downcasting with instanceof.

PROBLEM 3 • Intermediate

Quarterly Bonus Calculator

Problem Statement

A company calculates a year-end bonus for every staff member, but the formula genuinely differs by role. Separately, some roles must also produce an audit record for compliance — a requirement that has nothing to do with how bonuses are calculated.

Requirements

- StaffMember must be an abstract class with an abstract method calculateBonus() and a private baseSalary field exposed only through JavaBean getSalary()/setSalary() methods — setSalary() must reject a negative value instead of applying it.
- StaffMember must provide two constructors: one taking just baseSalary (defaulting an internal bonusRate to 0.10), and one taking both baseSalary and bonusRate — the simpler constructor must chain to the richer one using this(...).
- Define a separate interface Auditable with a method auditRecord() returning a summary string.
- TeamLead must extend StaffMember and implement Auditable, providing its own calculateBonus() (using the inherited bonusRate) and its own auditRecord(); its constructor must reach StaffMember's constructor via super(...).
- Write a method that accepts a StaffMember reference, checks with instanceof whether it also happens to implement Auditable, and if so, safely casts it to call auditRecord() — otherwise, report that no audit is required.

Function Signature(s)

```
public abstract class StaffMember(double baseSalary)  // chains via this(baseSalary, 0.10)
public abstract class StaffMember(double baseSalary, double bonusRate)
public abstract double calculateBonus();
double getSalary()
void setSalary(double baseSalary)
public interface Auditable { String auditRecord(); }
public TeamLead(double baseSalary, int teamSize)
public TeamLead(double baseSalary, double bonusRate, int teamSize)
static String getAuditIfApplicable(StaffMember s)
```

Examples

Input	Output
`TeamLead t = new TeamLead(60000, 5);` `t.calculateBonus()`	`6000.0`

Explanation: *TeamLead(60000, 5) chains to StaffMember's this(baseSalary, 0.10) default, so the bonus is 10% of 60000.*

Input	Output
`TeamLead t2 = new TeamLead(60000, 0.20, 5);` `t2.calculateBonus()`	`12000.0`

Explanation: *Supplying an explicit bonusRate reaches StaffMember's two-argument constructor directly instead of the default.*

Input	Output
`t.setSalary(-5000)`	`rejected, salary unchanged`

Input	Output

`StaffMember ref = t; // upcasting: TeamLead stored as its parent type`
`getAuditIfApplicable(ref)`
	`"TeamLead audit: 5 team members, salary $60000.0"`

HINT The instanceof check here tests against Auditable, not against a specific class — that's what lets the same method work for any future class that adds the Auditable capability later, not just TeamLead. For the constructors, let the one-argument version do nothing but call this(baseSalary, 0.10) — don't repeat the field assignment in both places.

CONSTRAINTS: baseSalary is always non-negative at construction.

Key Concepts Tested: Abstract classes, interfaces implemented alongside inheritance, constructor overloading with this() chaining, super() reaching a parent constructor, encapsulation via JavaBean getters/setters, upcasting, instanceof against an interface type.

PROBLEM 4 • Intermediate → Advanced

Universal Media Launcher

Problem Statement

A media app wants to play anything 'playable' — audio files that are genuine media files with real shared properties, and podcasts, which behave like media from the outside but are actually streamed live and don't fit the same mold at all.

Requirements

- Define an interface Playable with play() and pause() methods, plus an overloaded play(int fromSecond) — true compile-time polymorphism, since it shares a name with play() but takes a different parameter list.
- MediaFile must be an abstract class with a final fileId assigned via a shared static counter, and an abstract method getFormatInfo(); AudioFile must extend MediaFile and implement Playable, providing both play() and play(int fromSecond).
- Podcast must implement Playable directly (both overloads), with no relationship to MediaFile at all — it manages its own episode data independently.
- Write a static method launchAll(Playable[] items) that loops through a mixed array and calls the no-argument play() on each polymorphically.
- Be ready to justify: why is AudioFile designed as a MediaFile subclass (IS-A) while Podcast is only a Playable (CAN-DO), given that both are 'played' the same way from the outside?

Function Signature(s)

```
public interface Playable { String play(); String play(int fromSecond); String pause(); }
public abstract class MediaFile  // assigns a final fileId internally
public abstract String getFormatInfo();
String getFileId()
public AudioFile(String title)
public Podcast(String showName, int episodeNumber)
static void launchAll(Playable[] items)
```

Examples

Input	Output

`AudioFile a = new AudioFile("Morning Jazz");`
`a.play()`
`a.play(30)`
`a.getFormatInfo()`
	`"Playing audio: Morning Jazz"`
	`"Playing audio: Morning Jazz from 0:30"`
	`"Audio file, ID: MF-1001"`

Explanation: *play() and play(int fromSecond) are two separate methods that happen to share a name — Java picks the right one based on how many arguments are passed, at compile time.*

Input	Output
`Podcast p = new Podcast("Tech Talk", 12);` `p.play()`	`"Streaming episode 12 of Tech Talk"`

Input	Output
`Playable ref = a; // upcasting: AudioFile reference stored as the Playable interface type`	`"Playing audio: Morning Jazz"`

Input	Output
`launchAll(new Playable[]{ ref, p })`	`"Streaming episode 12 of Tech Talk"`

HINT Ask what each class would lose if it were forced into the other's shape — would Podcast gain anything from a fileId and a getFormatInfo() it has no use for? That answer is the actual justification, not just "because the problem says so." For the overload, let play(int fromSecond) build its message however it likes — it doesn't need to call play() internally unless you want it to.

CONSTRAINTS: episodeNumber is always positive.

Key Concepts Tested: Interface vs abstract class roles, IS-A (extends) vs CAN-DO (implements) reasoning, compile-time polymorphism via overloaded interface methods, final fields from a static counter, upcasting, polymorphic dispatch over an interface-typed array.

PROBLEM 5 • Advanced

Community Library Checkout System

Problem Statement

A community library checks out physical textbooks and issues digital passes for e-resources. Textbooks can be renewed and reserved by other members; digital passes can only ever be renewed, and don't belong to the library's physical item hierarchy at all.

Requirements

- LibraryItem must be an abstract class with a final itemId assigned via a shared static counter, and an abstract method getLoanPeriodDays().
- Define two interfaces: Renewable (renew()) and Reservable (reserve()).
- Textbook must extend LibraryItem and implement BOTH Renewable and Reservable.
- Magazine must also extend LibraryItem (hierarchical — a sibling of Textbook) but implement only Renewable, not Reservable, showing that siblings under the same parent can still adopt different capabilities.
- DigitalPass must implement only Renewable, with no relationship to LibraryItem whatsoever.
- Write a static method processCheckouts(LibraryItem[] items) that loops through an array of LibraryItem references and prints each one's loan period polymorphically.
- Separately, write a method that accepts any Object, checks with instanceof whether it implements Reservable, and if so, safely casts it to call reserve() — otherwise, report that reservation isn't supported.

Function Signature(s)

```
public abstract class LibraryItem  // assigns a final itemId internally
public abstract int getLoanPeriodDays();
String getItemId()
public interface Renewable { String renew(); }
public interface Reservable { String reserve(); }
public Textbook(String title)
public Magazine(String title)
public DigitalPass(String resourceName)
static void processCheckouts(LibraryItem[] items)
static String reserveIfSupported(Object o)
```

Examples

Input	Output

`Textbook t = new Textbook("Java Fundamentals");`
`t.getLoanPeriodDays()`
`t.renew()`
`t.reserve()`
	`14`
	`"Java Fundamentals renewed"`
	`"Java Fundamentals reserved"`

Input	Output
`Magazine m = new Magazine("Tech Monthly");` `reserveIfSupported(m)`	`"Reservation not supported"`

Explanation: *Magazine extends the same LibraryItem parent as Textbook, but choosing not to implement Reservable is exactly the point — sharing a parent never forces two siblings to share the same capabilities.*

Input	Output
`DigitalPass d = new DigitalPass("E-Journal Access");` `reserveIfSupported(d)`	`"Reservation not supported"`

Input	Output

`LibraryItem ref = t; // upcasting: Textbook stored as its parent type`
`reserveIfSupported(ref)`
	`"Java Fundamentals reserved"`

HINT reserveIfSupported(Object o) takes a plain Object on purpose — it needs to work on anything, whether or not it's even a LibraryItem, since Reservable and LibraryItem are completely independent of each other. Magazine and Textbook only need to differ in their implements clause — nothing about LibraryItem itself needs to change to allow that.

CONSTRAINTS: title and resourceName are never blank.

Key Concepts Tested: Abstract classes, hierarchical inheritance where siblings adopt different interfaces, a class implementing multiple interfaces at once, IS-A vs CAN-DO independence, upcasting, polymorphic dispatch, instanceof against an interface on a general Object reference.